Beyond the Chatbot: Building LLMs into Business Systems

Demonstrating how structured LLM outputs feed real-world technology endpoints

Model: **openai/gpt-oss-120b** | Generated: 2026-03-10 19:17:29 UTC | Triage Routing: **SURGERY**

This showcase demonstrates eight concrete integration patterns where a validated, schema-constrained `Triage` object — produced by the ER Intake Triage structured-output LLM pipeline — feeds directly into real-world technology endpoints used by healthcare, government, academic, and insurance organizations. Each card below shows the target system, the specific Pydantic fields consumed, live values extracted from the current analysis, and realistic integration code.

1 Hospital EHR — FHIR Observation Resources

Healthcare | REST / FHIR R4 API

Push each extracted symptom as a FHIR Observation resource into the hospital's Electronic Health Records system, creating a structured clinical record from unstructured intake transcripts.

MAPPED FIELDS — LIVE VALUES

SymptomRead.clinical: "New holosystolic murmur radiating to axilla consistent with severe mitral regurgitation, persistent and worsening since early February", "Pulmonary edema with pink frothy sputum, present at rest and increasing, indicating acute fluid overload", "Severe crushing chest pressure, constant, escalating to 8-9/10, present at rest and with activity", "Hypotension with systolic BP dropping to 96 mmHg, progressive from baseline 128/78, indicating circulatory compromise", "Tachycardia with resting heart rate up to 124 bpm, persistent and irregular, worsening over days" ... and 9 more
SymptomRead.severity: range [2, 5], mean 3.6

SAMPLE INTEGRATION CODE

```
import requests, json

FHIR_BASE = class="py-string">"https://ehr.hospital.org/fhir/R4"

for calc in triage.calculations:
    for cluster in calc.clusters:
        for symptom in cluster.symptoms:
            observation = {
                class="py-string">"resourceType": class="py-string">"Observation",
                class="py-string">"status": class="py-string">"final",
                class="py-string">"category": [{
                    class="py-string">"coding": [{
                        class="py-string">"system": class="py-string">"http://terminology.hl7.org/CodeSystem/observation-category",
                        class="py-string">"code": class="py-string">"exam",
                        class="py-string">"display": class="py-string">"Exam"
                    }]
                }],
                class="py-string">"code": {
                    class="py-string">"text": symptom.clinical
                },
                class="py-string">"valueInteger": symptom.severity,
                class="py-string">"interpretation": [{
                    class="py-string">"text": fclass="py-string">"Severity {symptom.severity}/5"
                }]
            }
            resp = requests.post(
                fclass="py-string">"{FHIR_BASE}/Observation",
                json=observation,
                headers={class="py-string">"Content-Type": class="py-string">"application/fhir+json"},
                timeout=30,
            )
            resp.raise_for_status()
```

2 CDC Syndromic Surveillance — MQTT Stream

Government | MQTT Message Broker

Stream real-time acute cluster alerts to a public health syndromic surveillance system for early outbreak detection and situational awareness.

MAPPED FIELDS — LIVE VALUES

SymptomCluster.acute: 3 acute / 3 total clusters
SymptomCluster.symptoms (count): [5, 5, 4] (total 14)

SAMPLE INTEGRATION CODE

```
import paho.mqtt.client as mqtt
import json

client = mqtt.Client(client_id=class="py-string">"er-triage-pipeline")
client.tls_set()  class="py-comment"># Enable TLS for secure transport
client.connect(class="py-string">"mqtt.cdc.gov", port=8883)

for calc in triage.calculations:
    for cluster in calc.clusters:
        if cluster.acute:
            payload = json.dumps({
                class="py-string">"alert_type": class="py-string">"acute_cluster",
                class="py-string">"symptom_count": len(cluster.symptoms),
                class="py-string">"symptoms": [s.clinical for s in cluster.symptoms],
                class="py-string">"acute": cluster.acute,
                class="py-string">"timestamp": datetime.now(timezone.utc).isoformat(),
            })
            client.publish(
                class="py-string">"cdc/syndromic/acute_alerts",
                payload=payload,
                qos=1,
            )

client.disconnect()
```

3 Bed Management Dashboard — Redis Cache

Hospital Operations | Redis In-Memory Cache

Instantly update a real-time bed-management dashboard with the triage routing decision so that charge nurses and bed coordinators can allocate resources before the patient arrives at the unit.

MAPPED FIELDS — LIVE VALUES

Triage.judgment: SURGERY

SAMPLE INTEGRATION CODE

```
import redis, json

r = redis.Redis(host=class="py-string">"redis.hospital.local", port=6379, db=0)

patient_id = class="py-string">"patient-20260310-001"
routing = {
    class="py-string">"judgment": triage.judgment,
    class="py-string">"timestamp": datetime.now(timezone.utc).isoformat(),
    class="py-string">"priority": class="py-string">"STAT" if triage.judgment in (class="py-string">"surgery", class="py-string">"ICU") else class="py-string">"routine",
}

class="py-comment"># Cache routing decision with 2-hour TTL
r.set(
    fclass="py-string">"triage:routing:{patient_id}",
    json.dumps(routing),
    ex=7200,
)

class="py-comment"># Publish to real-time channel for dashboard subscribers
r.publish(class="py-string">"triage:routing", json.dumps({
    class="py-string">"patient_id": patient_id,
    **routing,
}))
```

4 Clinical Research Data Lake — Parquet/S3

Clinical Research | Apache Parquet / AWS S3 Data Lake

Write flattened risk calculation records as columnar Parquet files to a research data lake for retrospective cohort analysis and outcomes research.

MAPPED FIELDS — LIVE VALUES

RiskCalc.risk: 0.96, 0.88, 0.80
RiskCalc[0].clusters: 1 clusters (1 acute)
RiskCalc[1].clusters: 1 clusters (1 acute)
RiskCalc[2].clusters: 1 clusters (1 acute)

SAMPLE INTEGRATION CODE

```
import pyarrow as pa
import pyarrow.parquet as pq

rows = []
for i, calc in enumerate(triage.calculations):
    acute_count = sum(1 for c in calc.clusters if c.acute)
    rows.append({
        class="py-string">"calc_index": i,
        class="py-string">"risk_score": calc.risk,
        class="py-string">"cluster_count": len(calc.clusters),
        class="py-string">"acute_cluster_count": acute_count,
        class="py-string">"judgment": triage.judgment,
        class="py-string">"timestamp": datetime.now(timezone.utc).isoformat(),
    })

table = pa.Table.from_pylist(rows)
pq.write_to_dataset(
    table,
    root_path=class="py-string">"s3://research-datalake/triage/risk_calcs",
    partition_cols=[class="py-string">"judgment"],
)
```

5 Insurance Pre-Authorization — SOAP/XML

Insurance | SOAP / XML Web Service

Submit the highest risk score and triage routing to a payer's pre-authorization service for automated approval of emergent procedures, reducing administrative delays in critical cases.

MAPPED FIELDS — LIVE VALUES

RiskCalc.risk: 0.96, 0.88, 0.80
Triage.judgment: SURGERY

SAMPLE INTEGRATION CODE

```
from zeep import Client

wsdl = class="py-string">"https://payer.example.com/preauth?wsdl"
soap_client = Client(wsdl)

max_risk = max(calc.risk for calc in triage.calculations)
response = soap_client.service.SubmitPreAuthorization(
    PatientId=class="py-string">"patient-20260310-001",
    RiskScore=max_risk,
    TriageRouting=triage.judgment,
    Urgency=class="py-string">"emergent" if max_risk ≥ 0.8 else class="py-string">"urgent",
    RequestTimestamp=datetime.now(timezone.utc).isoformat(),
)

if response.Approved:
    auth_number = response.AuthorizationNumber
    print(fclass="py-string">"Pre-auth approved: {auth_number}")
```

6 Medical School Teaching Database — PostgreSQL

Academic | PostgreSQL Relational Database

Insert deduplicated symptom extractions into a normalized teaching database that medical students query to study symptom-to-diagnosis reasoning patterns.

MAPPED FIELDS — LIVE VALUES

SymptomRead.clinical: "New holosystolic murmur radiating to axilla consistent with severe mitral regurgitation, persistent and worsening since early February", "Pulmonary edema with pink frothy sputum, present at rest and increasing, indicating acute fluid overload", "Severe crushing chest pressure, constant, escalating to 8-9/10, present at rest and with activity", "Hypotension with systolic BP dropping to 96 mmHg, progressive from baseline 128/78, indicating circulatory compromise", "Tachycardia with resting heart rate up to 124 bpm, persistent and irregular, worsening over days" ... and 9 more
SymptomRead.severity: range [2, 5], mean 3.6
SymptomCluster.acute: 3 acute / 3 total clusters

SAMPLE INTEGRATION CODE

```
import psycopg2

conn = psycopg2.connect(
    host=class="py-string">"db.medschool.edu", dbname=class="py-string">"teaching_cases",
    user=class="py-string">"etl_service", password=class="py-string">"***",
)
cur = conn.cursor()

try:
    for calc in triage.calculations:
        for cluster in calc.clusters:
            cur.execute(
                class="py-string">"INSERT INTO clusters (acute, symptom_count, created_at) "
                class="py-string">"VALUES (%s, %s, NOW()) RETURNING id",
                (cluster.acute, len(cluster.symptoms)),
            )
            cluster_id = cur.fetchone()[0]

            for symptom in cluster.symptoms:
                cur.execute(
                    class="py-string">"INSERT INTO symptoms (cluster_id, clinical, severity) "
                    class="py-string">"VALUES (%s, %s, %s)",
                    (cluster_id, symptom.clinical, symptom.severity),
                )
    conn.commit()
finally:
    cur.close()
    conn.close()
```

7 Emergency Department Event Stream — Kafka

Emergency Services | Apache Kafka Event Stream

Publish triage routing events with risk metadata to a Kafka topic so that downstream systems (alerting, analytics, staffing) can react in real time to incoming patient acuity.

MAPPED FIELDS — LIVE VALUES

Triage.judgment: SURGERY
RiskCalc.risk (max): 0.96

SAMPLE INTEGRATION CODE

```
from confluent_kafka import Producer
import json, socket

producer = Producer({
    class="py-string">"bootstrap.servers": class="py-string">"kafka.hospital.local:9092",
    class="py-string">"client.id": socket.gethostname(),
})

max_risk = max(calc.risk for calc in triage.calculations)
event = json.dumps({
    class="py-string">"event_type": class="py-string">"triage_routing",
    class="py-string">"judgment": triage.judgment,
    class="py-string">"max_risk_score": max_risk,
    class="py-string">"risk_dimensions": len(triage.calculations),
    class="py-string">"timestamp": datetime.now(timezone.utc).isoformat(),
})

producer.produce(
    topic=class="py-string">"ed.triage.events",
    key=class="py-string">"patient-20260310-001",
    value=event,
)
producer.flush(timeout=10)
```

8 State Health Department Analytics — GraphQL

Public Health | GraphQL API

Report aggregated cluster-level data to a state health department's analytics platform for population health monitoring, resource planning, and epidemiological trend detection.

MAPPED FIELDS — LIVE VALUES

SymptomCluster.symptoms: "New holosystolic murmur radiating to axilla consistent with severe mitral regurgitation, persistent and worsening since early February", "Pulmonary edema with pink frothy sputum, present at rest and increasing, indicating acute fluid overload", "Severe crushing chest pressure, constant, escalating to 8-9/10, present at rest and with activity", "Hypotension with systolic BP dropping to 96 mmHg, progressive from baseline 128/78, indicating circulatory compromise", "Tachycardia with resting heart rate up to 124 bpm, persistent and irregular, worsening over days", "Dyspnea with orthopnea and paroxysmal nocturnal dyspnea, worsening to inability to lie flat, requiring multiple pillows" ... and 8 more
SymptomCluster.acute: 3 acute / 3 total clusters
RiskCalc.risk: 0.96, 0.88, 0.80

SAMPLE INTEGRATION CODE

```
from gql import gql, Client
from gql.transport.requests import RequestsHTTPTransport

transport = RequestsHTTPTransport(
    url=class="py-string">"https://analytics.health.state.gov/graphql",
    headers={class="py-string">"Authorization": "Bearer <token>class="py-string">"},
)
gql_client = Client(transport=transport, fetch_schema_from_transport=True)

mutation = gql(class="py-string">"class="py-string">""
    mutation CreateTriageReport($input: TriageReportInput!) {
        createTriageReport(input: $input) {
            id
            status
        }
    }
class="py-string">""class="py-string">")

clusters_data = []
for calc in triage.calculations:
    for cluster in calc.clusters:
        clusters_data.append({
            "symptomsclass="py-string">": [s.clinical for s in cluster.symptoms],
            "acuteclass="py-string">": cluster.acute,
            "symptomCountclass="py-string">": len(cluster.symptoms),
        })

result = gql_client.execute(mutation, variable_values={
    "inputclass="py-string">": {
        "clustersclass="py-string">": clusters_data,
        "riskScoresclass="py-string">": [c.risk for c in triage.calculations],
        "judgment": triage.judgment,
    }
})
```

Educational Demonstration — This showcase is part of the "Beyond the Chatbot: Building LLMs into Business Systems" presentation. All integration endpoints shown are simulated for illustration purposes. The structured data values displayed above are real outputs from the ER Intake Triage pipeline using the **openai/gpt-oss-120b** model.